Sumario

1 - Introducción y Estructura de un archivo CSV....3
¿Qué es un archivo CSV?....3
Diferencias con otros formatos....3
Usos más comunes....3
Estructura de un archivo CSV....4
Separadores más frecuentes....4
Codificación de texto....4
Encabezados y filas de datos....5
Limitaciones del formato....5
2 - Creación y Edición de CSV....5
Crear un CSV manualmente con un editor de texto....5
Guardar un Excel como CSV....6
Herramientas online para generar CSV....6
3 - Lectura de CSV en Python y Java....7
Lectura de CSV en Python....7
Usando el módulo csv (Python estándar)....7
Usando pandas (para análisis de datos)....8
Lectura de CSV en Java....8
Usando Scanner (Java estándar)....8
Usando OpenCSV (biblioteca de terceros)....9
4 - Lectura de CSV en C# y JavaScript, y Escritura de CSV....11
Lectura de CSV en C#....11
Lectura de CSV en JavaScript....12
Usando PapaParse (navegador y Node.js)....12
Usando FileReader (navegador, para archivos locales)....13
Escritura de CSV....14
Crear un CSV desde un programa....14
Exportar datos a CSV desde una base de datos....15
Generar reportes automatizados....15
5 - Manejo de Datos Especiales en CSV....16
Campos con comillas y separadores internos....16
Manejo de saltos de línea dentro de una celda....16
Datos nulos o vacíos....17
Diferentes codificaciones y cómo detectarlas....17
Codificaciones comunes:....17
Cómo detectar la codificación:....17
6 - Operaciones con CSV en Python (detalle con Pandas)....18
Lectura con pandas.read_csv()....18
Filtrado y selección de columnas....19
Agregar y modificar registros....19
Guardar con to_csv()....20
7 - Validación, Limpieza y CSV con Bases de Datos....20
Validación y limpieza de CSV....20
Detectar y corregir errores comunes....20
Verificar consistencia de columnas....21
Eliminar duplicados....21

Normalización de datos....21
CSV y bases de datos....22
Importar CSV a MySQL, PostgreSQL y SQLite....22
Exportar tablas a CSV....22
Uso en ETL (Extract, Transform, Load)....23
8 - Buenas Prácticas y Ejercicios Prácticos con CSV....23
Buenas prácticas....23
Documentar el separador y la codificación....23
Usar nombres de columnas claros....24
Evitar ambigüedades en datos....24
Versionado y control de cambios en CSV grandes....24
Ejercicios prácticos....24
Ejercicio 1: Crear un CSV de productos....24
Ejercicio 2: Analizar un dataset clásico (ejemplo: Titanic en Kaggle)....25
Ejercicio 3: Exportar resultados de un análisis a CSV....25
Mini proyecto: Aplicación simple que lea, filtre y guarde CSV....25

1 - Introducción y Estructura de un archivo CSV

¿Qué es un archivo CSV?

Un archivo CSV (Comma Separated Values o Valores Separados por Comas) es un formato de archivo de texto plano que utiliza comas para separar los valores. Cada línea del archivo representa una fila de datos, y cada valor dentro de una línea es un campo de esa fila. Es un formato muy simple y ampliamente utilizado para el intercambio de datos tabulares entre diferentes aplicaciones.

A pesar de su nombre, los archivos CSV no siempre usan comas como separador; pueden usar otros caracteres como punto y coma, tabulaciones, etc., dependiendo de la configuración regional o la aplicación que los genera.

Diferencias con otros formatos

- **Excel (XLSX/XLS):** Los archivos de Excel son binarios y pueden contener múltiples hojas, formatos complejos, fórmulas, macros y gráficos. Un CSV es texto plano, solo contiene datos y no tiene formato ni funcionalidades avanzadas. Excel puede abrir y guardar CSV, pero al hacerlo, se pierde toda la información de formato y las hojas adicionales.
- **TSV (Tab Separated Values):** Similar a CSV, pero utiliza tabulaciones en lugar de comas como separador de campos. Es útil cuando los datos pueden contener comas internamente, evitando conflictos sin necesidad de comillas.
- **JSON (JavaScript Object Notation):** JSON es un formato de intercambio de datos estructurados que utiliza una sintaxis basada en objetos y arreglos de JavaScript. Es más flexible que CSV para representar datos jerárquicos o anidados, mientras que CSV es ideal para datos tabulares simples.

Usos más comunes

- **Ciencia de Datos y Análisis:** Es el formato preferido para almacenar y compartir datasets pequeños y medianos debido a su simplicidad y facilidad de procesamiento con herramientas como Pandas en Python.
- **Programación:** Se utiliza para importar/exportar configuraciones, logs, o datos de prueba en aplicaciones.
- **Negocios:** Comúnmente usado para exportar informes de ventas, listas de clientes, datos de inventario desde sistemas de gestión o bases de datos.
- **Migración de Datos:** Una forma sencilla de mover datos entre diferentes sistemas o bases de datos.

Estructura de un archivo CSV

La estructura básica de un archivo CSV es muy sencilla:

```
columna1,columna2,columna3
valor1_1,valor1_2,valor1_3
valor2_1,valor2_2,valor2_3
```

- Cada línea es un registro de datos.
- Los campos dentro de cada registro están separados por un delimitador (comúnmente una coma).
- La primera línea a menudo contiene los nombres de las columnas (encabezados), aunque no es obligatorio.

Separadores más frecuentes

- **Coma (`,`):** El separador más común y el que da nombre al formato. Ejemplo: `nombre,edad,ciudad`
- **Punto y coma (`;`):** Frecuentemente usado en regiones donde la coma es el separador decimal (ej. muchos países europeos y latinoamericanos) para evitar ambigüedades. Ejemplo: `nombre;edad;ciudad`
- **Tabulación (`\t`):** Utilizado en archivos TSV. Es útil cuando los datos pueden contener comas o puntos y comas. Ejemplo: `nombre\tedad\tciudad`
- **Otros:** Ocasionalmente se pueden encontrar otros separadores como el pipe (`|`) o el circunflejo (`^`), pero son menos comunes.

Codificación de texto

La codificación de texto define cómo los caracteres son representados en bytes. Es crucial para que los caracteres especiales (acentos, eñes, etc.) se muestren correctamente.

- **UTF-8:** Es la codificación más recomendada y universal. Soporta todos los caracteres de todos los idiomas y es compatible con ASCII. Debería ser la opción por defecto siempre que sea posible.
- **ANSI (Windows-1252):** Una codificación común en sistemas Windows, especialmente en archivos CSV generados por Excel en versiones antiguas. Puede causar problemas con caracteres especiales si se abre en sistemas que esperan UTF-8.
- **ISO-8859-1 (Latin-1):** Similar a ANSI, cubre la mayoría de los caracteres de Europa Occidental. También puede generar problemas de compatibilidad.

Es importante saber qué codificación usa un archivo CSV para leerlo correctamente y evitar caracteres extraños.

Encabezados y filas de datos

- **Encabezados:** La primera línea de un archivo CSV a menudo contiene los nombres de las columnas. Esto facilita la comprensión de los datos y su procesamiento programático, ya que se puede referenciar los datos por su nombre en lugar de por su índice numérico.
- **Filas de Datos:** Las líneas subsiguientes contienen los registros de datos. Cada valor en una fila corresponde a la columna definida por el encabezado (si existe).

Limitaciones del formato

- **No jerárquico:** CSV es plano; no puede representar datos anidados o estructuras complejas como JSON o XML.
- **Sin tipos de datos explícitos:** Todos los datos se almacenan como texto. La interpretación del tipo (número, fecha, booleano) debe hacerse al leer el archivo.
- **Manejo de caracteres especiales:** Si un campo contiene el carácter delimitador (ej. una coma en un campo separado por comas) o saltos de línea, debe ser encerrado entre comillas dobles (`"`). Esto puede complicar la lectura si no se implementa correctamente.
- **Sin metadatos:** No hay forma estándar de incluir información sobre el archivo en sí (autor, fecha de creación, etc.) dentro del CSV.

2 - Creación y Edición de CSV

Crear un CSV manualmente con un editor de texto

La forma más básica de crear un archivo CSV es utilizando un editor de texto plano como Notepad (Windows), TextEdit (macOS), VS Code, Sublime Text, o cualquier otro. Simplemente abres un nuevo archivo y sigues la estructura de campos separados por un delimitador y registros separados por saltos de línea.

Pasos:

1. Abre tu editor de texto preferido.
2. Escribe los encabezados de las columnas en la primera línea, separados por comas (o el delimitador que vayas a usar).
3. En las líneas siguientes, introduce los datos, asegurándote de que cada valor corresponda a su columna y esté separado por el mismo delimitador.
4. Guarda el archivo con la extensión `.csv`. Asegúrate de seleccionar la codificación correcta (preferiblemente UTF-8).

Ejemplo:

```
Nombre,Edad,Ciudad
Juan,30,Madrid
Ana,24,Barcelona
Carlos,35,Valencia
```

Guardar un Excel como CSV

Microsoft Excel (y otras hojas de cálculo como Google Sheets o LibreOffice Calc) es una de las herramientas más comunes para trabajar con datos tabulares y, por lo tanto, para generar archivos CSV.

Pasos en Excel:

1. Abre tu archivo de Excel (`.xlsx` o `.xls`).
2. Asegúrate de que los datos que quieres exportar estén en una sola hoja.
3. Ve a `Archivo > Guardar como`.
4. En el cuadro de diálogo `Guardar como`, selecciona la ubicación donde quieres guardar el archivo.
5. En el menú desplegable `Tipo` (o `Guardar como tipo`), elige `CSV (delimitado por comas) (*.csv)`.
6. Haz clic en `Guardar`. Es posible que Excel te advierta sobre la pérdida de características si el libro contiene múltiples hojas o formatos. Confirma para guardar solo la hoja activa como CSV.

Consideraciones:

- Excel usará el delimitador de lista configurado en la configuración regional de tu sistema operativo (coma o punto y coma).
- Los formatos de celda, fórmulas, gráficos y múltiples hojas no se guardarán en el archivo CSV.
- Si tienes caracteres especiales, asegúrate de que Excel guarde el archivo con la codificación UTF-8 si es posible (en versiones recientes de Excel, esto se puede seleccionar en las opciones de guardado).

Herramientas online para generar CSV

Existen varias herramientas online que te permiten generar o convertir datos a formato CSV, útiles para tareas rápidas o para quienes no tienen software de hoja de cálculo.

- **Convertidores de JSON a CSV:** Si tienes datos en formato JSON, muchas herramientas online te permiten convertirlos fácilmente a CSV. Por ejemplo, JSON to CSV Converter o ConvertCSV.com.
- **Generadores de datos de prueba:** Sitios como Mockaroo o GenerateData.com permiten crear grandes volúmenes de datos de prueba con formatos específicos, incluyendo CSV, para desarrollo y testing.
- **Editores CSV online:** Algunas plataformas ofrecen editores básicos para manipular archivos CSV directamente en el navegador, como CSVJSON que también permite editar y convertir.

Estas herramientas son convenientes para tareas puntuales, pero para manipulación de datos a gran escala o automatización, es preferible usar lenguajes de programación.

3 - Lectura de CSV en Python y Java

Leer archivos CSV es una tarea fundamental en el procesamiento de datos. A continuación, veremos cómo hacerlo en dos de los lenguajes de programación más populares: Python y Java.

Lectura de CSV en Python

Python ofrece varias formas de leer archivos CSV, siendo las más comunes el módulo `csv` integrado y la biblioteca `pandas`, muy utilizada en ciencia de datos.

Usando el módulo `csv` (Python estándar)

El módulo `csv` de Python permite leer y escribir archivos CSV de manera sencilla, manejando automáticamente las comillas y los delimitadores.

Ejemplo de archivo `datos.csv`:

```
Nombre,Edad,Ciudad
Juan,30,Madrid
Ana,24,"Barcelona, España"
Carlos,35,Valencia
```

Código Python para leer `datos.csv`:

```
import csv

with open('datos.csv', mode='r', encoding='utf-8') as file:
    csv_reader = csv.reader(file)
    header = next(csv_reader) # Lee la primera fila como encabezado
    print(f"Encabezado: {header}")
    for row in csv_reader:
        print(row)

# Para leer como diccionarios (más conveniente)
print("\nLeyendo como diccionarios:")
with open('datos.csv', mode='r', encoding='utf-8') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        print(row['Nombre'], row['Edad'], row['Ciudad'])
        # Acceso por nombre de columna
```

Explicación:

- `csv.reader`: Itera sobre las líneas del archivo CSV y devuelve cada línea como una lista de cadenas.
- `next(csv_reader)`: Se usa para leer la primera línea, que generalmente contiene los encabezados.

- `csv.DictReader`: Es una clase muy útil que lee cada fila como un diccionario, donde las claves son los nombres de las columnas (tomados de la primera fila). Esto facilita el acceso a los datos por nombre.
- `encoding='utf-8'`: Es crucial especificar la codificación para evitar problemas con caracteres especiales.

Usando `pandas` (para análisis de datos)

La biblioteca `pandas` es el estándar de facto en Python para el análisis y manipulación de datos. Proporciona estructuras de datos como `DataFrame` que son ideales para trabajar con datos tabulares.

Para usar pandas, primero necesitas instalarlo: `pip install pandas`

Código Python para leer `datos.csv` con pandas:

```
import pandas as pd

# Leer el archivo CSV
df = pd.read_csv('datos.csv', encoding='utf-8')

print("DataFrame completo:")
print(df)

print("\nAccediendo a una columna:")
print(df['Nombre'])

print("\nFiltrando datos (ej. personas mayores de 30):")
print(df[df['Edad'] > 30])
```

Explicación:

- `pd.read_csv()`: Es la función principal para leer archivos CSV. Es muy potente y tiene muchos parámetros para manejar delimitadores, encabezados, codificaciones, etc.
- El resultado es un `DataFrame`, que es una tabla bidimensional con etiquetas para filas y columnas.
- Acceder a columnas es sencillo usando `df['NombreColumna']`.
- Pandas facilita el filtrado, la selección y la manipulación de datos de manera eficiente.

Lectura de CSV en Java

En Java, puedes leer archivos CSV utilizando clases estándar de I/O o bibliotecas de terceros que ofrecen más funcionalidades.

Usando `Scanner` (Java estándar)

Para archivos CSV simples, puedes usar la clase `Scanner` para leer línea por línea y luego dividir cada línea.

Código Java para leer `datos.csv` con Scanner:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class LeerCsvScanner {
    public static void main(String[] args) {
        String fileName = "datos.csv";
        File file = new File(fileName);

        try (Scanner scanner = new Scanner(file, "UTF-8")) {
            // Leer encabezado
            if (scanner.hasNextLine()) {
                String headerLine = scanner.nextLine();
                String[] headers = headerLine.split(",");
                System.out.println("Encabezado: ");
                for (String h : headers) {
                    System.out.print(h + "\t");
                }
                System.out.println();
            }

            // Leer datos
            while (scanner.hasNextLine()) {
                String dataLine = scanner.nextLine();
                String[] data = dataLine.split(",");
                for (String d : data) {
                    System.out.print(d + "\t");
                }
                System.out.println();
            }

        } catch (FileNotFoundException e) {
            System.err.println("Archivo no encontrado: " + fileName);
        } catch (Exception e) {
            System.err.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}
```

Explicación:

- Se crea un `Scanner` para leer el archivo, especificando la codificación UTF-8.
- Se lee la primera línea como encabezado y se divide por comas.
- Luego, se itera sobre las líneas restantes, dividiendo cada una para obtener los campos.
- Este método es simple pero no maneja automáticamente campos con comas internas o comillas.

Usando `OpenCSV` (biblioteca de terceros)

Para un manejo más robusto de archivos CSV en Java, especialmente aquellos con campos complejos (comas internas, saltos de línea, comillas), se recomienda usar una biblioteca como OpenCSV.

Primero, necesitas añadir la dependencia a tu proyecto (ej. en Maven):

```
<dependency>
    <groupId>com.opencsv</groupId>
    <artifactId>opencsv</artifactId>
    <version>5.9</version> <!-- Usa la versión más reciente -->
</dependency>
```

Código Java para leer `datos.csv` con OpenCSV:

```
import com.opencsv.CSVReader;
import com.opencsv.exceptions.CsvException;

import java.io.FileReader;
import java.io.IOException;
import java.util.List;

public class LeerCsvOpenCsv {
    public static void main(String[] args) {
        String fileName = "datos.csv";

        try (CSVReader reader = new CSVReader(new FileReader(fileName))) {
            List<String[]> allRecords = reader.readAll();

            // Imprimir encabezado
            if (!allRecords.isEmpty()) {
                String[] header = allRecords.get(0);
                System.out.println("Encabezado: ");
                for (String h : header) {
                    System.out.print(h + "\t");
                }
                System.out.println();

                // Imprimir datos
                for (int i = 1; i < allRecords.size(); i++) {
                    String[] record = allRecords.get(i);
                    for (String field : record) {
                        System.out.print(field + "\t");
                    }
                    System.out.println();
                }
            }

        } catch (IOException | CsvException e) {
            System.err.println("Error al leer el archivo CSV: " +
e.getMessage());
        }
    }
}
```

Explicación:

- `CSVReader`: Proporciona métodos para leer archivos CSV de forma robusta.
- `readAll()`: Lee todo el archivo y devuelve una `List` de `String[]`, donde cada `String[]` es una fila.

- OpenCSV maneja automáticamente las comillas, los delimitadores y los saltos de línea dentro de los campos, lo que lo hace más fiable que `Scanner` para CSV complejos.

4 - Lectura de CSV en C# y JavaScript, y Escritura de CSV

Lectura de CSV en C#

Para C#, la biblioteca `CsvHelper` es una opción muy popular y robusta para leer y escribir archivos CSV. Es altamente configurable y maneja bien los casos complejos.

Primero, instala el paquete NuGet `CsvHelper`:

```
dotnet add package CsvHelper
```

Ejemplo de archivo `datos.csv`:

```
Id,Producto,Precio
1,Manzana,1.20
2,Pera,0.85
3,"Naranja, dulce",1.50
```

Código C# para leer `datos.csv` con CsvHelper:

```
using CsvHelper;
using CsvHelper.Configuration;
using System.Globalization;

public class Producto
{
    public int Id { get; set; }
    public string Producto { get; set; }
    public decimal Precio { get; set; }
}

public class CsvReaderExample
{
    public static void Main(string[] args)
    {
        var config = new CsvConfiguration(CultureInfo.InvariantCulture)
        {
            HasHeaderRecord = true,
        };

        using (var reader = new StreamReader("datos.csv"))
        using (var csv = new CsvReader(reader, config))
        {
            var records = csv.GetRecords<Producto>().ToList();
            foreach (var p in records)
            {
```

```
                Console.WriteLine($"Id: {p.Id}, Producto: {p.Producto}, Precio:
{p.Precio}");
            }
        }
    }
}
```

Explicación:

- Se define una clase `Producto` que mapea las columnas del CSV.
- `CsvConfiguration` permite configurar el delimitador, si hay encabezados, etc. `CultureInfo.InvariantCulture` es importante para asegurar que los números se parseen correctamente independientemente de la configuración regional.
- `csv.GetRecords<Producto>()` lee automáticamente el CSV y lo mapea a una lista de objetos `Producto`.

Lectura de CSV en JavaScript

En JavaScript, la lectura de CSV puede variar si se hace en el navegador (usando `FileReader` para archivos locales o `fetch` para URLs) o en Node.js (usando módulos de sistema de archivos). Para ambos entornos, bibliotecas como `PapaParse` simplifican mucho la tarea.

Usando `PapaParse` (navegador y Node.js)

PapaParse es una potente biblioteca para parsear CSV en JavaScript, manejando automáticamente delimitadores, comillas, y conversiones de tipos.

Puedes incluirla vía CDN en HTML:

```
<script src="https://unpkg.com/papaparse@5.4.1/papaparse.min.js"></script>
```

Ejemplo de lectura de un archivo local (HTML + JS):

```
<input type="file" id="csvFile" accept=".csv">
<pre id="output"></pre>

<script>
  document.getElementById('csvFile').addEventListener('change', function(event)
{
    const file = event.target.files[0];
    Papa.parse(file, {
      header: true,
      complete: function(results) {
        document.getElementById('output').textContent =
JSON.stringify(results.data, null, 2);
        console.log("Parsed CSV:", results.data);
      },
      error: function(err) {
        console.error("Error parsing CSV:", err);
      }
    });
  });
```

```
</script>
```

Explicación:

- El usuario selecciona un archivo CSV a través de un input de tipo `file`.
- `Papa.parse()` toma el archivo y un objeto de configuración. `header: true` indica que la primera fila son encabezados.
- El callback `complete` se ejecuta cuando el parseo termina, y `results.data` contiene los datos como un array de objetos.

Usando `FileReader` (navegador, para archivos locales)

Para archivos CSV muy simples y sin dependencias externas, puedes usar la API `FileReader` del navegador.

```
<input type="file" id="csvFileSimple" accept=".csv">
<pre id="outputSimple"></pre>

<script>
  document.getElementById('csvFileSimple').addEventListener('change',
function(event) {
    const file = event.target.files[0];
    const reader = new FileReader();

    reader.onload = function(e) {
      const text = e.target.result;
      const lines = text.split('\n');
      const headers = lines[0].split(',');
      const data = [];

      for (let i = 1; i < lines.length; i++) {
        if (lines[i].trim() === '') continue; // Ignorar líneas vacías
        const values = lines[i].split(',');
        const row = {};
        for (let j = 0; j < headers.length; j++) {
          row[headers[j]] = values[j];
        }
        data.push(row);
      }
      document.getElementById('outputSimple').textContent = JSON.stringify(data,
null, 2);
      console.log("Parsed CSV (simple):
", data);
    };

    reader.readAsText(file);
  });
</script>
```

Explicación:

- `FileReader` lee el contenido del archivo como texto.
- El contenido se divide por saltos de línea y luego cada línea por comas.

- Este método es más propenso a errores si el CSV no es perfectamente simple (ej. comas dentro de campos).

Escritura de CSV

La escritura de archivos CSV es el proceso inverso a la lectura: tomar datos estructurados (desde un programa, una base de datos, etc.) y convertirlos en el formato de texto plano CSV.

Crear un CSV desde un programa

La mayoría de los lenguajes de programación ofrecen módulos o bibliotecas para escribir datos en formato CSV.

Ejemplo en Python (usando el módulo `csv`):

```
import csv

datos = [
    ['Nombre', 'Edad', 'Ciudad'],
    ['Pedro', 28, 'Sevilla'],
    ['Maria', 32, 'Malaga'],
    ['Luis', 45, '"Granada, España"' ] # Campo con coma interna
]

with open('salida.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerows(datos)

print("Archivo salida.csv creado exitosamente.")
```

Explicación:

- `newline=''` es importante para evitar que se inserten líneas en blanco adicionales en Windows.
- `csv.writer` crea un objeto escritor.
- `writerows()` escribe todas las filas de una lista de listas. El módulo `csv` se encarga automáticamente de las comillas si un campo contiene el delimitador.

Ejemplo en Java (usando OpenCSV):

```
import com.opencsv.CSVWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class EscribirCsvOpenCsv {
    public static void main(String[] args) {
        String fileName = "salida.csv";

        List<String[]> datos = new ArrayList<>();
        datos.add(new String[]{"Nombre", "Edad", "Ciudad"});
        datos.add(new String[]{"Pedro", "28", "Sevilla"});
```

```
        datos.add(new String[]{"Maria", "32", "Malaga"});
        datos.add(new String[]{"Luis", "45", "Granada, España"});

        try (CSVWriter writer = new CSVWriter(new FileWriter(fileName))) {
            writer.writeAll(datos);
            System.out.println("Archivo " + fileName + " creado exitosamente.");
        } catch (IOException e) {
            System.err.println("Error al escribir el archivo CSV: " +
e.getMessage());
        }
    }
}
```

Explicación:

- `CSVWriter` se encarga de formatear los datos correctamente, incluyendo el manejo de comillas.
- `writeAll()` escribe una lista de arrays de cadenas al archivo.

Exportar datos a CSV desde una base de datos

La mayoría de los sistemas de gestión de bases de datos (DBMS) ofrecen funcionalidades para exportar los resultados de una consulta directamente a un archivo CSV.

- **MySQL:** Puedes usar la sentencia `SELECT ... INTO OUTFILE`.

  ```
  SELECT id, nombre, email
  FROM usuarios
  INTO OUTFILE '/tmp/usuarios.csv'
  FIELDS TERMINATED BY ','
  ENCLOSED BY '"'
  LINES TERMINATED BY '\n';
  ```

- **PostgreSQL:** Utiliza el comando `COPY`.

  ```
  COPY (SELECT id, nombre, email FROM usuarios) TO '/tmp/usuarios.csv' WITH
  CSV HEADER;
  ```

- **SQLite:** Desde la línea de comandos, puedes configurar el modo CSV y luego ejecutar una consulta.

  ```
  sqlite3 database.db
  .mode csv
  .headers on
  .output usuarios.csv
  SELECT id, nombre, email FROM usuarios;
  .quit
  ```

Generar reportes automatizados

La capacidad de escribir CSV programáticamente es fundamental para la generación de reportes automatizados. Por ejemplo, un script Python puede:

1. Conectarse a una base de datos.

2. Ejecutar una consulta compleja para obtener datos.
3. Procesar y transformar esos datos.
4. Escribir los resultados en un archivo CSV.
5. Enviar el archivo CSV por correo electrónico o subirlo a un servidor FTP.

Esto es muy común en tareas de ETL (Extract, Transform, Load) y en la creación de dashboards o informes periódicos para negocios.

5 - Manejo de Datos Especiales en CSV

Los archivos CSV, a pesar de su simplicidad, pueden presentar desafíos cuando los datos contienen caracteres especiales como comas, saltos de línea o comillas. Entender cómo manejar estos casos es crucial para evitar errores de parseo.

Campos con comillas y separadores internos

Cuando un campo de datos contiene el carácter delimitador (por ejemplo, una coma en un archivo CSV delimitado por comas) o comillas dobles, el estándar CSV indica que todo el campo debe encerrarse entre comillas dobles. Si el campo ya contiene comillas dobles, estas deben ser "escapadas" duplicándolas.

Ejemplo:

```
Producto,Descripción,Precio
"Manzana","Una fruta roja, dulce y crujiente",1.50
"Libro","""El Quijote"", una obra maestra",20.00
Naranja,Fruta cítrica,1.20
```

- La descripción de la manzana contiene una coma, por lo que todo el campo está entre comillas dobles.
- La descripción del libro contiene comillas dobles (para "El Quijote"), por lo que esas comillas internas se duplican (`"""El Quijote"""`) y todo el campo se encierra entre comillas dobles.

La mayoría de las bibliotecas de parseo de CSV manejan esto automáticamente, pero es importante ser consciente de esta convención.

Manejo de saltos de línea dentro de una celda

Un campo CSV también puede contener saltos de línea. Para que el parser no interprete el salto de línea como el final de una fila, el campo completo debe estar encerrado entre comillas dobles.

Ejemplo:

```
ID,Comentario
1,"Este es un comentario
con múltiples líneas."
```

```
2,Comentario de una sola línea.
```

Aquí, el comentario para el ID 1 se extiende a lo largo de dos líneas en el archivo CSV, pero se considera un único campo debido a las comillas dobles que lo encierran.

Datos nulos o vacíos

Los datos nulos o vacíos en CSV se representan generalmente de dos maneras:

- **Campo vacío:** Simplemente se deja el espacio entre delimitadores sin ningún valor. Por ejemplo, `valor1,,valor3` indica que el segundo campo está vacío.
- **Campo con un valor específico para nulo:** Algunas aplicaciones utilizan cadenas como `NULL`, `NA`, `N/A` o `-` para representar valores nulos. Es importante conocer la convención utilizada en el archivo para interpretarlos correctamente.

Ejemplo:

```
Nombre,Edad,Email
Juan,30,juan@example.com
Ana,,ana@example.com
Carlos,40,
```

En este ejemplo, Ana tiene un campo de edad vacío, y Carlos tiene un campo de email vacío.

Diferentes codificaciones y cómo detectarlas

Como se mencionó en la introducción, la codificación de caracteres es fundamental. Si un archivo CSV se lee con la codificación incorrecta, los caracteres especiales (acentos, eñes, símbolos) aparecerán como símbolos extraños o ilegibles (mojibake).

Codificaciones comunes:

- **UTF-8:** La codificación universal y recomendada. Soporta todos los caracteres y es compatible con ASCII.
- **ISO-8859-1 (Latin-1):** Común en Europa Occidental.
- **Windows-1252 (ANSI):** Una extensión de ISO-8859-1, muy usada en sistemas Windows.

Cómo detectar la codificación:

- **Preguntar al origen:** La forma más fiable es preguntar a quien generó el archivo qué codificación utilizó.
- **Probar:** Si no se conoce, se puede intentar abrir el archivo con diferentes codificaciones (UTF-8, luego ISO-8859-1/Windows-1252) hasta que los caracteres se muestren correctamente.
- **Herramientas de detección:** Existen herramientas y bibliotecas que intentan adivinar la codificación. Por ejemplo, en Python, la biblioteca `chardet` puede ser útil:

  ```
  import chardet
  ```

```
with open('archivo.csv', 'rb') as rawdata:
    result = chardet.detect(rawdata.read(10000)) # Lee los primeros 10KB

print(result) # {'encoding': 'UTF-8', 'confidence': 0.99, 'language': ''}
```

- **Editores de texto avanzados:** Muchos editores de texto (VS Code, Sublime Text, Notepad++) pueden detectar y mostrar la codificación de un archivo.

Siempre que sea posible, se debe estandarizar el uso de UTF-8 para evitar estos problemas.

6 - Operaciones con CSV en Python (detalle con Pandas)

La biblioteca `pandas` es una herramienta indispensable en Python para el análisis y manipulación de datos tabulares, incluyendo archivos CSV. Ofrece una gran flexibilidad y eficiencia para realizar diversas operaciones.

Para los ejemplos, asumiremos que tenemos un archivo `productos.csv` con el siguiente contenido:

```
ID,Nombre,Categoría,Precio,Stock
1,Laptop,Electrónica,1200.50,10
2,Teclado,Electrónica,75.00,25
3,Mouse,Electrónica,25.99,50
4,Monitor,Electrónica,300.00,15
5,Mesa,Muebles,150.00,5
6,Silla,Muebles,80.00,20
```

Lectura con `pandas.read_csv()`

La función `read_csv()` es la puerta de entrada para cargar datos CSV en un DataFrame de pandas. Es muy potente y configurable.

```
import pandas as pd

# Lectura básica
df = pd.read_csv('productos.csv')
print("DataFrame completo:\n", df)

# Especificando delimitador (si no es coma)
df_semicolon = pd.read_csv('productos_semicolon.csv', sep=';')

# Especificando codificación
df_latin1 = pd.read_csv('productos_latin1.csv', encoding='latin1')

# Seleccionar solo algunas columnas al leer
df_subset = pd.read_csv('productos.csv', usecols=['Nombre', 'Precio'])
print("\nDataFrame con columnas seleccionadas:\n", df_subset)

# Leer sin encabezado (si el archivo no tiene)
```

```
df_no_header = pd.read_csv('productos_no_header.csv', header=None,
names=['Col1', 'Col2', 'Col3'])
print("\nDataFrame sin encabezado:\n", df_no_header)
```

Filtrado y selección de columnas

Una vez que los datos están en un DataFrame, pandas facilita enormemente la selección de columnas y el filtrado de filas.

```
# Seleccionar una sola columna
nombres = df['Nombre']
print("\nNombres de productos:\n", nombres)

# Seleccionar múltiples columnas
productos_precios = df[['Nombre', 'Precio']]
print("\nNombres y precios:\n", productos_precios)

# Filtrar filas por una condición (productos con precio > 100)
productos_caros = df[df['Precio'] > 100]
print("\nProductos caros:\n", productos_caros)

# Filtrar por múltiples condiciones (categoría Electrónica y stock < 20)
electronica_bajo_stock = df[(df['Categoría'] == 'Electrónica') & (df['Stock'] <
20)]
print("\nElectrónica con bajo stock:\n", electronica_bajo_stock)

# Filtrar usando .loc (selección por etiqueta)
fila_id_2 = df.loc[df['ID'] == 2]
print("\nFila con ID 2:\n", fila_id_2)

# Filtrar usando .iloc (selección por posición entera)
primera_fila = df.iloc[0]
print("\nPrimera fila:\n", primera_fila)
```

Agregar y modificar registros

Pandas permite agregar nuevas filas y modificar valores existentes de manera intuitiva.

```
# Modificar un valor específico (cambiar el precio del Mouse)
df.loc[df['Nombre'] == 'Mouse', 'Precio'] = 29.99
print("\nDataFrame con precio de Mouse modificado:\n", df)

# Modificar múltiples valores (aumentar stock de Electrónica en 5)
df.loc[df['Categoría'] == 'Electrónica', 'Stock'] += 5
print("\nDataFrame con stock de Electrónica aumentado:\n", df)

# Agregar una nueva columna (ej. 'ValorTotal')
df['ValorTotal'] = df['Precio'] * df['Stock']
print("\nDataFrame con columna 'ValorTotal':\n", df)

# Agregar una nueva fila (usando pd.concat)
nuevo_producto = pd.DataFrame([{'ID': 7, 'Nombre': 'Cama', 'Categoría':
'Muebles', 'Precio': 400.00, 'Stock': 8}])
df = pd.concat([df, nuevo_producto], ignore_index=True)
print("\nDataFrame con nueva fila:\n", df)
```

Guardar con `to_csv()`

Una vez que has manipulado los datos, puedes guardarlos de nuevo en un archivo CSV usando la función `to_csv()`.

```
# Guardar el DataFrame modificado a un nuevo archivo CSV
df.to_csv('productos_actualizados.csv', index=False, encoding='utf-8')
print("\nDataFrame guardado en productos_actualizados.csv")

# Guardar con un delimitador diferente
df.to_csv('productos_semicolon_out.csv', sep=';', index=False, encoding='utf-8')

# Guardar solo algunas columnas
df[['Nombre', 'ValorTotal']].to_csv('reporte_valor_total.csv', index=False,
encoding='utf-8')
```

Explicación:

- `index=False`: Es muy importante para evitar que pandas escriba el índice del DataFrame como una columna adicional en el CSV.
- `encoding='utf-8'`: Asegura que el archivo se guarde con la codificación correcta.
- Puedes especificar el delimitador con el parámetro `sep`.

7 - Validación, Limpieza y CSV con Bases de Datos

Trabajar con datos CSV a menudo implica lidiar con imperfecciones. La validación y limpieza son pasos cruciales para asegurar la calidad de los datos antes de su análisis o importación a sistemas.

Validación y limpieza de CSV

Los datos en archivos CSV rara vez son perfectos. Pueden contener errores de entrada, inconsistencias, duplicados o formatos incorrectos. La limpieza de datos es el proceso de corregir o eliminar estos errores.

Detectar y corregir errores comunes

- **Valores faltantes:** Identificar celdas vacías o con marcadores de nulo (`NA`, `NULL`). Se pueden rellenar con un valor por defecto, la media/mediana, o eliminar la fila/columna.

  ```
  import pandas as pd

  df = pd.read_csv('datos_sucios.csv')
  print("Valores nulos antes:\n", df.isnull().sum())

  # Rellenar nulos en la columna 'Edad' con la media
  df['Edad'].fillna(df['Edad'].mean(), inplace=True)
  ```

```
# Eliminar filas con cualquier valor nulo
df.dropna(inplace=True)
print("Valores nulos después:\n", df.isnull().sum())
```

- **Tipos de datos incorrectos:** Asegurarse de que las columnas numéricas sean números, las fechas sean fechas, etc. A menudo, los números pueden venir como cadenas con caracteres no numéricos.

```
# Convertir columna a tipo numérico, forzando errores a NaN
df['Precio'] = pd.to_numeric(df['Precio'], errors='coerce')
# Eliminar filas donde la conversión falló
df.dropna(subset=['Precio'], inplace=True)
```

- **Espacios en blanco extra:** Eliminar espacios al inicio o final de las cadenas.

```
df['Nombre'] = df['Nombre'].str.strip()
```

Verificar consistencia de columnas

Asegurarse de que los valores en una columna sigan un patrón o un conjunto de valores permitidos.

- **Valores únicos/categorías:** Verificar que una columna categórica solo contenga los valores esperados.

```
print("Categorías únicas:\n", df['Categoría'].unique())
# Corregir inconsistencias (ej. 'Electronica' y 'electrónica')
df['Categoría'] = df['Categoría'].replace({'Electronica': 'Electrónica'})
```

- **Rangos de valores:** Para columnas numéricas, verificar que los valores estén dentro de un rango lógico (ej. edad > 0 y < 120).

Eliminar duplicados

Los registros duplicados pueden sesgar los análisis. Es importante identificarlos y decidir si eliminarlos o no.

```
print("Filas duplicadas antes:\n", df.duplicated().sum())
# Eliminar filas completamente duplicadas
df.drop_duplicates(inplace=True)

# Eliminar duplicados basándose en un subconjunto de columnas (ej. ID de
producto)
df.drop_duplicates(subset=['ID'], inplace=True)
```

Normalización de datos

Transformar los datos a un formato estándar. Esto puede incluir:

- Convertir texto a minúsculas o mayúsculas.
- Estandarizar formatos de fecha.
- Escalar valores numéricos (ej. a un rango de 0 a 1).

CSV y bases de datos

Los archivos CSV son un formato muy común para interactuar con bases de datos, tanto para importar datos existentes como para exportar resultados de consultas.

Importar CSV a MySQL, PostgreSQL y SQLite

La mayoría de los sistemas de bases de datos relacionales tienen comandos o herramientas para importar datos desde archivos CSV.

- **MySQL:** Utiliza el comando `LOAD DATA INFILE`.

  ```
  LOAD DATA INFILE '/path/to/data.csv'
  INTO TABLE my_table
  FIELDS TERMINATED BY ','
  ENCLOSED BY '"'
  LINES TERMINATED BY '\n'
  IGNORE 1 ROWS; -- Para ignorar la fila de encabezado
  ```

- **PostgreSQL:** Utiliza el comando `COPY`.

  ```
  COPY my_table FROM '/path/to/data.csv' WITH (FORMAT CSV, HEADER TRUE);
  ```

- **SQLite:** Desde la línea de comandos, puedes usar `.import`.

  ```
  .mode csv
  .import /path/to/data.csv my_table
  ```

- **Python con Pandas:** Puedes leer el CSV con pandas y luego usar `to_sql()` para insertarlo en una base de datos (requiere una librería de conexión a la DB, ej. `sqlalchemy` y `psycopg2` para PostgreSQL).

  ```
  from sqlalchemy import create_engine

  # Conexión a PostgreSQL (ejemplo)
  engine = create_engine('postgresql://user:password@host:port/database')

  df = pd.read_csv('datos.csv')

  # Escribir el DataFrame a una tabla en la base de datos
  df.to_sql('nombre_tabla', engine, if_exists='replace', index=False)
  ```

Exportar tablas a CSV

De manera similar, puedes exportar datos de una tabla o el resultado de una consulta a un archivo CSV.

- **MySQL:** `SELECT ... INTO OUTFILE` (ya visto en Tema 4).
- **PostgreSQL:** `COPY ... TO` (ya visto en Tema 4).
- **SQLite:** Usando el modo CSV en la CLI (ya visto en Tema 4).
- **Python con Pandas:** Leer desde la base de datos y luego usar `to_csv()`.

```
# Leer datos de una tabla de base de datos a un DataFrame
df_db = pd.read_sql('SELECT * FROM nombre_tabla', engine)

# Guardar el DataFrame a CSV
df_db.to_csv('export_db.csv', index=False, encoding='utf-8')
```

Uso en ETL (Extract, Transform, Load)

Los archivos CSV juegan un papel fundamental en los procesos ETL:

- **Extract (Extracción):** Los datos se extraen de diversas fuentes (bases de datos, APIs, sistemas legados) y a menudo se guardan temporalmente como CSV debido a su simplicidad y universalidad.
- **Transform (Transformación):** Los datos CSV se cargan en herramientas o scripts (como Python con pandas) donde se limpian, validan, normalizan, agregan y transforman según las necesidades del negocio.
- **Load (Carga):** Los datos transformados se cargan finalmente en un sistema de destino, que puede ser un data warehouse, una base de datos analítica o una aplicación, a menudo utilizando los mismos mecanismos de importación de CSV.

La simplicidad y el formato de texto plano de CSV lo hacen un formato de intercambio ideal en muchas etapas de un pipeline ETL.

8 - Buenas Prácticas y Ejercicios Prácticos con CSV

Para finalizar este tutorial, abordaremos algunas buenas prácticas que te ayudarán a trabajar de manera más eficiente y a evitar problemas comunes con los archivos CSV, seguidas de algunos ejercicios prácticos para consolidar lo aprendido.

Buenas prácticas

Seguir estas recomendaciones te permitirá crear y manejar archivos CSV de forma más robusta y compatible.

Documentar el separador y la codificación

Siempre que compartas o generes un archivo CSV, es una buena práctica documentar qué delimitador (coma, punto y coma, tabulación) y qué codificación (UTF-8, ISO-8859-1) se utilizaron. Esto evita problemas de interpretación en el receptor.

- Puedes incluir esta información en el nombre del archivo (ej. `datos_productos_utf8_coma.csv`).
- O en un archivo `README.txt` adjunto.

- Idealmente, usar siempre UTF-8 y coma como separador, a menos que haya una razón específica para no hacerlo.

Usar nombres de columnas claros

Los encabezados de las columnas deben ser descriptivos, concisos y no contener caracteres especiales o espacios si es posible (usar guiones bajos _ en su lugar). Esto facilita el procesamiento programático y la comprensión humana.

- **Mal:** `Nombre Cliente`, `Fecha de Venta`
- **Bien:** `nombre_cliente`, `fecha_venta`

Evitar ambigüedades en datos

- **Fechas:** Utiliza un formato de fecha estándar e inequívoco, como `YYYY-MM-DD` (ej. `2023-10-27`). Evita formatos como `MM/DD/YY` o `DD/MM/YY` que pueden ser confusos.
- **Números:** Si usas decimales, asegúrate de que el separador decimal sea consistente (punto `.` es el estándar en programación, pero la coma `,` es común en algunas regiones). Si usas coma como separador decimal, entonces el delimitador de campos no debería ser la coma.
- **Valores booleanos:** Usa `true/false`, `1/0` o `T/F` de forma consistente.

Versionado y control de cambios en CSV grandes

Para archivos CSV que cambian con frecuencia o son parte de un proyecto colaborativo, considera usar sistemas de control de versiones como Git. Aunque Git no está optimizado para archivos binarios grandes, para CSV de tamaño razonable puede ser útil para rastrear cambios. Para CSV muy grandes, considera soluciones de versionado de datos (Data Version Control - DVC) o bases de datos.

Ejercicios prácticos

Aplica lo aprendido con estos ejercicios. Puedes usar Python con pandas o el lenguaje de tu elección.

Ejercicio 1: Crear un CSV de productos

Crea un script que genere un archivo CSV llamado `productos_inventario.csv` con las siguientes columnas y al menos 5 registros:

- `id_producto` (entero)
- `nombre_producto` (cadena)
- `categoria` (cadena, ej. "Electrónica", "Alimentos", "Hogar")
- `precio_unitario` (decimal)
- `stock_disponible` (entero)
- `fecha_ultima_revision` (fecha en formato YYYY-MM-DD)

Asegúrate de que al menos un producto tenga un nombre con una coma interna (ej. "Televisor, Smart TV") y que se maneje correctamente con comillas.

Ejercicio 2: Analizar un dataset clásico (ejemplo: Titanic en Kaggle)

Descarga el dataset `titanic.csv` de Kaggle (puedes buscar "Titanic Dataset Kaggle" y descargar el archivo `train.csv`). Luego, escribe un script que:

1. Cargue el archivo `titanic.csv` en un DataFrame de pandas.
2. Muestre las primeras 5 filas y la información general del DataFrame (`.info()`, `.describe()`).
3. Calcule el número de sobrevivientes y no sobrevivientes.
4. Calcule la tasa de supervivencia por género.
5. Encuentre la edad promedio de los pasajeros.
6. Maneje los valores nulos en la columna 'Age' (puedes rellenarlos con la media o mediana).
7. Guarde un nuevo CSV llamado `titanic_analisis.csv` con las columnas `Survived`, `Pclass`, `Sex`, `Age`, `Fare`, y la nueva columna `FamilySize` (sumando `SibSp` y `Parch`).

Ejercicio 3: Exportar resultados de un análisis a CSV

Partiendo del ejercicio anterior, supongamos que tienes un DataFrame con los resultados de un análisis (ej. la tasa de supervivencia por clase y género). Escribe un script que:

1. Cree un DataFrame de pandas con datos ficticios o reales de un análisis (ej. `Clase,Genero,Tasa_Supervivencia`).
2. Exporte este DataFrame a un archivo CSV llamado `reporte_supervivencia.csv`.
3. Asegúrate de que el archivo no incluya el índice del DataFrame.

Mini proyecto: Aplicación simple que lea, filtre y guarde CSV

Desarrolla una pequeña aplicación (puede ser en Python con una interfaz de línea de comandos simple) que permita al usuario:

1. Especificar la ruta de un archivo CSV de entrada.
2. Especificar una columna y un valor para filtrar las filas (ej. filtrar productos por `categoria == 'Electrónica'`).
3. Especificar las columnas que desea mantener en el archivo de salida.
4. Guardar los resultados filtrados y seleccionados en un nuevo archivo CSV de salida.

Este mini proyecto integrará la lectura, el filtrado, la selección y la escritura de CSV, consolidando gran parte de lo aprendido en el tutorial.